kind and Google Pub/Sub

A practical guide to running a local Kubernetes cluster with kind and testing Google Cloud Pub/Sub locally or against a real Google Cloud project.

What is kind?

kind (Kubernetes IN Docker) runs a Kubernetes cluster in Docker containers. It is useful for local development and testing; it does not create Google Cloud resources. Pub/Sub can be tested locally with its emulator or reached from a kind workload using Google Cloud credentials.

Prerequisites

Install Docker, kind, kubectl, and make. For the real Google Cloud example, install and configure the Google Cloud CLI (`gcloud`).

Run commands in a terminal from this project directory (`kinD/`).

Start a kind cluster

Create the default cluster and select its Kubernetes context:

```
kind create cluster --name kind
kubectl config use-context kind-kind
kubectl get nodes
```

The cluster can be deleted when no longer needed:

```
kind delete cluster --name kind
```

Basic kind example

The included `Dockerfile` builds a small BusyBox image that prints a greeting. Build and load the image into kind:

```
docker build -t my-custom-app:1.0 .
kind load docker-image my-custom-app:1.0 --name kind
```

Apply the example pod and check its output:

```
kubectl apply -f pod.yaml
kubectl get pods
kubectl logs test-pod
```

The pod exits after printing its message, so `Completed` is an expected status.

Pub/Sub overview

Pub/Sub has three main concepts:

- **Topic:** publishers send messages to a topic.
- **Subscription:** connects a consumer to a topic and retains messages for it.
- **Publisher and consumer:** client applications (represented here by pods), not Pub/Sub resources.

Each subscription gets its own copy of messages. Consumers sharing one subscription compete to receive its messages.

Local Pub/Sub emulator

The emulator runs in kind and does not access Google Cloud. Start the emulator and its administration, publisher, and consumer pods:

```
make pubsub-emulator-up
kubectl get pods
```

Create a topic and a subscription

Create the topic first, then attach a subscription for the consumer:

```
kubectl exec pubsub-emulator-client -- curl -sS -X PUT \
  'http://pubsub-emulator:8085/v1/projects/kind-pubsub/topics/demo'

kubectl exec pubsub-emulator-client -- curl -sS -X PUT \
  'http://pubsub-emulator:8085/v1/projects/kind-pubsub/subscriptions/demo-sub' \
  -H 'Content-Type: application/json' \
  -d '{"topic":"projects/kind-pubsub/topics/demo"}'
```

Publish, pull, and acknowledge a message

Send a base64-encoded message from the publisher pod:

```
kubectl exec pubsub-publisher -- curl -sS -X POST \
  'http://pubsub-emulator:8085/v1/projects/kind-pubsub/topics/demo:publish' \
  -H 'Content-Type: application/json' \
  -d '{"messages":[{"data":"SGVsbG8gZnJvbSB0aGUgcHVibGlzaGVyIQ=="}]}'
```

The data decodes to `Hello from the publisher!`. Pull messages from the subscription in the consumer pod:

```
kubectl exec pubsub-consumer -- curl -sS -X POST \
  'http://pubsub-emulator:8085/v1/projects/kind-pubsub/subscriptions/demo-sub:pull' \
  -H 'Content-Type: application/json' \
  -d '{"maxMessages":10}'
```

The response includes the message and an `ackId`. Acknowledge it after processing; replace the example value with the `ackId` returned by the pull:

```
kubectl exec pubsub-consumer -- curl -sS -X POST \
  'http://pubsub-emulator:8085/v1/projects/kind-pubsub/subscriptions/demo-sub:acknowledge' \
  -H 'Content-Type: application/json' \
  -d '{"ackIds":["ACK_ID_FROM_PULL_RESPONSE"]}'
```

If a message is not acknowledged before its deadline, it can be delivered again. The emulator keeps its state only while its pod exists. Stop the emulator and demo clients with:

```
make pubsub-emulator-down
```

Connect to real Google Cloud Pub/Sub

This option creates and accesses real Google Cloud resources and may incur charges. kind is not GKE and does not provide Google workload identity. For a local test, use a dedicated service account and a key stored only on your machine and in a Kubernetes Secret. Never commit or share the key. For production on GKE, use Workload Identity Federation instead of a key.

Set the project, enable the API, create a topic and subscription, and create a service account for the publisher/consumer example:

```
gcloud config set project YOUR_PROJECT_ID
gcloud services enable pubsub.googleapis.com
gcloud pubsub topics create demo
gcloud pubsub subscriptions create demo-sub --topic=demo

PROJECT_ID="$(gcloud config get-value project)"
SERVICE_ACCOUNT="kind-pubsub-client@${PROJECT_ID}.iam.gserviceaccount.com"
gcloud iam service-accounts create kind-pubsub-client --project="$PROJECT_ID"
```

```
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/pubsub.publisher"
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/pubsub.subscriber"
gcloud iam service-accounts keys create "$HOME/kind-pubsub-credentials.json" \
  --iam-account="$SERVICE_ACCOUNT" \
  --project="$PROJECT_ID"
```

Create a ConfigMap for the project ID and a Secret for the service-account key. The key file path is assigned to an environment variable for the command:

```
export GOOGLE_APPLICATION_CREDENTIALS="$HOME/kind-pubsub-credentials.json"
kubectl create configmap pubsub-gcp-config \
  --from-literal=project-id="$(gcloud config get-value project)"
kubectl create secret generic gcp-pubsub-credentials \
  --from-file=credentials.json="$GOOGLE_APPLICATION_CREDENTIALS"
```

Deploy the authenticated client pod:

```
make pubsub-gcp-client
kubectl get pod pubsub-gcp-client
```

Publish a message and pull from the subscription. `--auto-ack` acknowledges messages as they are pulled:

```
kubectl exec pubsub-gcp-client -- gcloud pubsub topics publish demo \
  --message="Hello from kind" \
  --project="$(gcloud config get-value project)"

kubectl exec pubsub-gcp-client -- gcloud pubsub subscriptions pull demo-sub \
  --auto-ack --limit=10 \
  --project="$(gcloud config get-value project)"
```

Remove the client pod, Secret, and ConfigMap after testing. This does not delete the Google Cloud topic, subscription, service account, or key:

```
kubectl delete -f k8s/pubsub/gcp-client.yaml
kubectl delete secret gcp-pubsub-credentials
kubectl delete configmap pubsub-gcp-config
```

Delete cloud resources separately when appropriate, following your project's normal change controls.

Project files

- `Dockerfile`, `pod.yaml`: basic kind image and pod example.
- `Makefile`: kind and Pub/Sub helper commands (`make help` lists targets).
- `k8s/pubsub/emulator.yaml`: local Pub/Sub emulator Deployment and Service.
- `k8s/pubsub/emulator-client.yaml`: emulator administration pod.
- `k8s/pubsub/emulator-publisher-consumer.yaml`: publisher and consumer pods.
- `k8s/pubsub/gcp-client.yaml`: client pod for real Google Cloud Pub/Sub.
- `PUBSUB.md`: detailed Pub/Sub setup and commands.